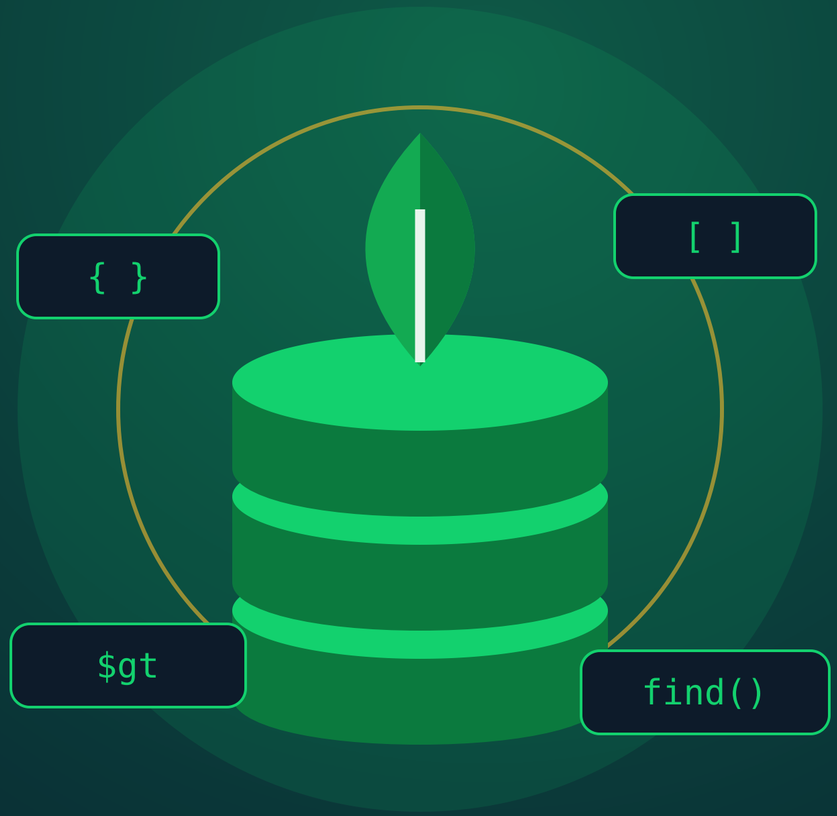
BIG DATA COURSE

SECTION 4

MongoDB Shell, Queries and Node.js API

mongosh · operators · arrays · update · drivers · Express · Postman · indexes · Atlas



Prepared by: Amira Mahmoud

SECTION 4

Roadmap: what we cover today

A

Shell and CRUD

mongosh, insert, find, sort, delete, update

B

Smart queries

nested fields, operators, $in, arrays

C

Node.js API

driver, Express, Postman

D

Scale and cloud

pagination, indexes, Atlas

Section 3 covered Compass. Section 4 repeats the same CRUD, now with code.

SECTION 4

Our toolbox

MongoDB Server
The database that stores the data (mongod)

MongoDB Compass
The visual app: click and see documents

mongosh
The shell: type commands, get results

Node.js
Runs JavaScript on the server for our API

Postman
Sends test requests to our API

MongoDB Atlas
MongoDB in the cloud, free tier

Section 3 used Compass. Section 4 adds code: mongosh, Node.js and Postman, then indexes and the cloud.

SECTION 4

Our data for today: 8 students

Name	Age	Major	GPA	City	Courses
Sara Ahmed	20	Computer Science	3.6	Cairo	SQL, DSP, Big Data, AI
Ali Hassan	22	Data Science	3.2	Alexandria	SQL, Python
Mona Khaled	21	Computer Science	3.9	Cairo	Big Data, AI, ML
Omar Said	23	IT	2.8	Giza	Networks
Nour Adel	20	IT	3.4	Mansoura	Networks, SQL
Youssef Tarek	22	Data Science	3.7	Cairo	Python, Big Data, ML
Hana Mostafa	21	Computer Science	3.1	Alexandria	DSP, AI
Karim Fathy	24	IT	2.5	Giza	Networks

> Same students as Section 3. We will load them again with one command.

SECTION 4

What is mongosh?

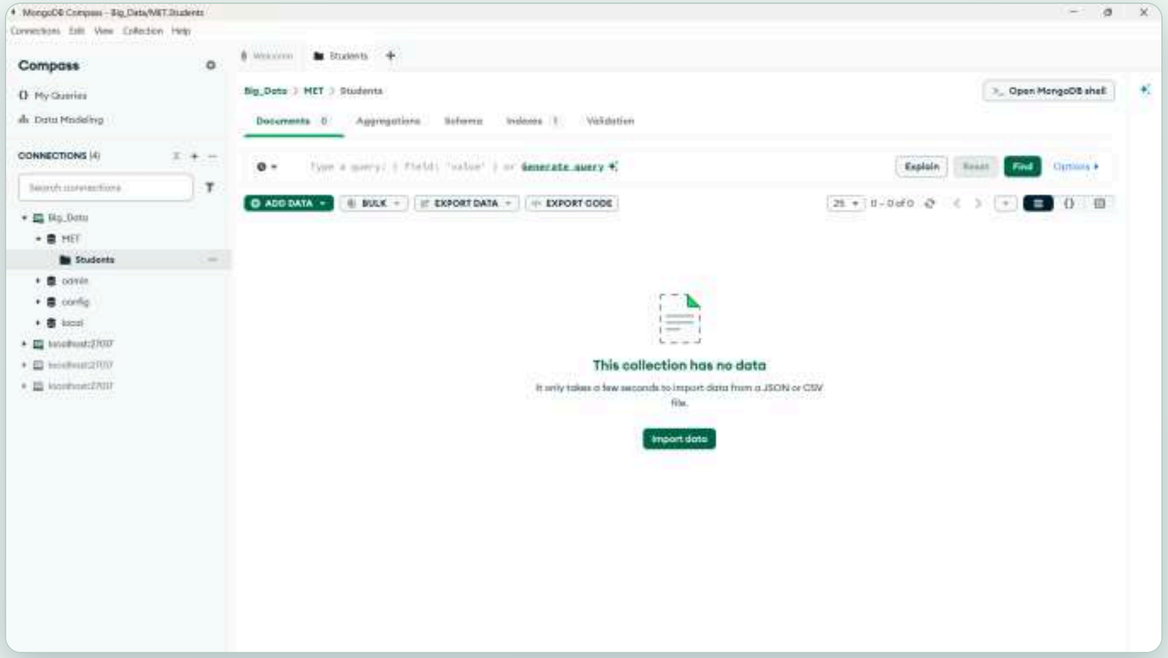
Compass
click and see

mongosh
type commands

→

mongod
one server, port 27017

1. Open a terminal and type `mongosh`
2. Or in Compass click **Open MongoDB shell** (top right)



Compass: Open MongoDB shell button, top right

SECTION 4

First commands in the shell

```
COMMANDS
mongosh
show dbs
use MET
show collections
db
help
exit
```

mongosh

```
test> show dbs
admin   40.00 KiB
config 108.00 KiB
local   72.00 KiB
MET     72.00 KiB
test> use MET
switched to db MET
test> show collections
students
```

Example output

Command	Meaning
`show dbs`	list databases
`use MET`	switch to database MET (created when data is saved)
`show collections`	list collections
`db`	name of the current database

SECTION 4

Add documents: insertOne

COMMAND

```
db.students.drop()

db.students.insertOne({
  name: "Sara Ahmed", age: 20,
  major: "Computer Science", gpa: 3.6,
  courses: ["SQL", "DSP", "Big Data", "AI"],
  address: { city: "Cairo", country: "Egypt" }
})
```

mongosh

```
MET> db.students.drop()
true
MET> db.students.insertOne({ name: "Sara Ahmed", ... })
{
  acknowledged: true,
  insertedId: ObjectId('65f1a2b3c4d5e6f7a8b9c001')
}
```

Example output

`db` is the current database, `students` the collection, `insertOne` the operation. The JSON document goes inside the parentheses.

SECTION 4

Add many: insertMany with 7 students

COMMAND · an array of documents inside insertMany

```
db.students.insertMany([
  { name: "Ali Hassan", age: 22, major: "Data Science", gpa: 3.2, courses: ["SQL", "Python"], address: { city: "Alexandria", country: "Egypt" } },
  { name: "Mona Khaled", age: 21, major: "Computer Science", gpa: 3.9, courses: ["Big Data", "AI", "ML"], address: { city: "Cairo", country: "Egypt" } },
  { name: "Omar Said", age: 23, major: "IT", gpa: 2.8, courses: ["Networks"], address: { city: "Giza", country: "Egypt" } },
  { name: "Nour Adel", age: 20, major: "IT", gpa: 3.4, courses: ["Networks", "SQL"], address: { city: "Mansoura", country: "Egypt" } },
  { name: "Youssef Tarek", age: 22, major: "Data Science", gpa: 3.7, courses: ["Python", "Big Data", "ML"], address: { city: "Cairo", country: "Egypt" } },
  { name: "Hana Mostafa", age: 21, major: "Computer Science", gpa: 3.1, courses: ["DSP", "AI"], address: { city: "Alexandria", country: "Egypt" } },
  { name: "Karim Fathy", age: 24, major: "IT", gpa: 2.5, courses: ["Networks"], address: { city: "Giza", country: "Egypt" } }
])
```

mongosh

```
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('65f1a2b3...c002'), '1': ObjectId('65f1a2b3...c003'), ... '6': ObjectId('65f1a2b3...c008') }
}
```

Example output

SECTION 4

Find documents: find and findOne

COMMANDS

```
db.students.countDocuments()

db.students.find({ major: "IT" })

db.students.find(
  { major: "IT" },
  { name: 1, gpa: 1, _id: 0 }
)

db.students.findOne({ name: "Sara Ahmed" })
```

mongosh

```
MET> db.students.countDocuments()
8
MET> db.students.find({ major: "IT" }, { name: 1, gpa: 1, _id: 0 })
[
  { name: 'Omar Said', gpa: 2.8 },
  { name: 'Nour Adel', gpa: 3.4 },
  { name: 'Karim Fathy', gpa: 2.5 }
]
```

Example output

Part	Role
First { }	filter: which documents
Second { }	projection: which fields (1 show, 0 hide)
`findOne`	returns one document, not a list

SECTION 4

findOne: one full document

COMMAND

```
db.students.findOne({ name: "Sara Ahmed" })
```

`find()` returns a **cursor** (a list you walk through). With more than 20 results the shell shows 20 and prints: Type "it" for more.

mongosh

```
MET> db.students.findOne({ name: "Sara Ahmed" })
{
  _id: ObjectId('65f1a2b3c4d5e6f7a8b9c001'),
  name: 'Sara Ahmed',
  age: 20,
  major: 'Computer Science',
  gpa: 3.6,
  courses: [ 'SQL', 'DSP', 'Big Data', 'AI' ],
  address: { city: 'Cairo', country: 'Egypt' }
}
```

Example output

SECTION 4

Sort and limit

COMMANDS

```
db.students.find(
  {}, { name: 1, gpa: 1, _id: 0 }
).sort({ gpa: -1 }).limit(3)

db.students.find(
  {}, { name: 1, _id: 0 }
).sort({ name: 1 }).skip(2).limit(3)
```

mongosh

```
MET> db.students.find({}, { name: 1, gpa: 1, _id: 0 }).sort({ gpa:
-1 }).limit(3)
[
  { name: 'Mona Khaled', gpa: 3.9 },
  { name: 'Youssef Tarek', gpa: 3.7 },
  { name: 'Sara Ahmed', gpa: 3.6 }
]
```

Example output

Method	Meaning
`sort({ gpa: -1 })`	-1 high to low, 1 low to high
`limit(3)`	only the first 3
`skip(2)`	jump over the first 2 (used for pagination)

SECTION 4

Nested documents: dot notation

COMMAND · quotes are required

```
db.students.find(
  { "address.city": "Cairo" },
  { name: 1, "address.city": 1, _id: 0 }
)
```

`address` is a document inside the student. Use `address.city` with a dot to reach the field inside, and keep it between quotes.

mongosh

```
MET> db.students.find({ "address.city": "Cairo" }, { name: 1,
"address.city": 1, _id: 0 })
[
  { name: 'Sara Ahmed', address: { city: 'Cairo' } },
  { name: 'Mona Khaled', address: { city: 'Cairo' } },
  { name: 'Youssef Tarek', address: { city: 'Cairo' } }
]
```

Example output

SECTION 4

Comparison operators

Operator	Meaning
`$gt`	greater than
`$gte`	greater than or equal
`$lt`	less than
`$lte`	less than or equal
`$ne`	not equal

COMMANDS

```
db.students.find(
  { gpa: { $gte: 3.5 } },
  { name: 1, gpa: 1, _id: 0 }
)

db.students.countDocuments(
  { major: { $ne: "IT" } }
)
```

mongosh

```
MET> db.students.find({ gpa: { $gte: 3.5 } }, { name: 1, gpa: 1,
_id: 0 })
[
  { name: 'Sara Ahmed', gpa: 3.6 },
  { name: 'Mona Khaled', gpa: 3.9 },
  { name: 'Youssef Tarek', gpa: 3.7 }
]
MET> db.students.countDocuments({ major: { $ne: "IT" } })
5
```

Example output

SECTION 4

Combine conditions: $or and $and

COMMANDS

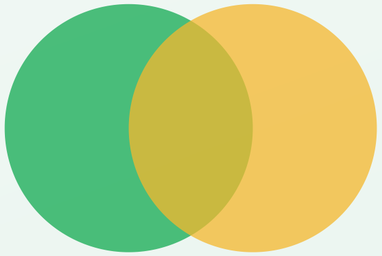
```
db.students.find(
  { $or: [
      { major: "IT" },
      { gpa: { $gte: 3.7 } }
  ] },
  { name: 1, major: 1, _id: 0 }
)

db.students.find({ $and: [
  { major: "IT" },
  { gpa: { $lt: 3 } }
] })
```

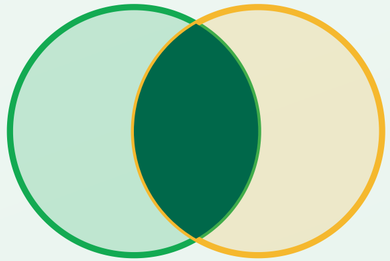
mongosh

```
MET> db.students.find({ $or: [ { major: "IT" }, { gpa: { $gte: 3.7 }
} ] }, { name: 1, major: 1, _id: 0 })
[
  { name: 'Mona Khaled', major: 'Computer Science' },
  { name: 'Omar Said', major: 'IT' },
  { name: 'Nour Adel', major: 'IT' },
  { name: 'Youssef Tarek', major: 'Data Science' },
  { name: 'Karim Fathy', major: 'IT' }
]
```

Example output



$or = either one



$and = both

Several fields in one filter are already AND. Use $and when the same field appears twice.

SECTION 4

$in and $nin

COMMANDS

```
db.students.find(
  { major: { $in: ["IT", "Data Science"] } },
  { name: 1, major: 1, _id: 0 }
)

db.students.find(
  { "address.city": { $nin: ["Cairo", "Alexandria"] } },
  { name: 1, "address.city": 1, _id: 0 }
)
```

`$in` = the value is one of the list. `$nin` = the value is not in the list.

mongosh

```
MET> ... { major: { $in: ["IT", "Data Science"] } } ...
[ Ali (Data Science), Omar (IT),
  Nour (IT), Youssef (Data Science),
  Karim (IT) ]

MET> ... { "address.city": { $nin: ["Cairo", "Alexandria"] } } ...
[
  { name: 'Omar Said', address: { city: 'Giza' } },
  { name: 'Nour Adel', address: { city: 'Mansoura' } },
  { name: 'Karim Fathy', address: { city: 'Giza' } }
]
```

Example output

SECTION 4

Querying arrays

COMMANDS

```
db.students.find({ courses: "SQL" }, { name: 1, _id: 0 })

db.students.find({ courses: ["Networks"] }, { name: 1, _id: 0 })

db.students.find(
  { courses: { $all: ["Big Data", "AI"] } },
  { name: 1, _id: 0 }
)
```

mongosh

```
MET> db.students.find({ courses: "SQL" }, { name: 1, _id: 0 })
[ { name: 'Sara Ahmed' }, { name: 'Ali Hassan' }, { name: 'Nour
Adel' } ]
MET> db.students.find({ courses: ["Networks"] }, { name: 1, _id: 0
})
[ { name: 'Omar Said' }, { name: 'Karim Fathy' } ]
MET> db.students.find({ courses: { $all: ["Big Data", "AI"] } }, {
name: 1, _id: 0 })
[ { name: 'Sara Ahmed' }, { name: 'Mona Khaled' } ]
```

Example output

Filter	Meaning
`{ courses: "SQL" }`	the array contains SQL
`{ courses: ["Networks"] }`	the array is exactly this list
`$all`	contains all of these values

SECTION 4

Arrays of documents: $elemMatch

COMMANDS

```
db.labs.insertMany([
  { student: "Sara", scores: [ { lab: "L1", mark: 9 }, { lab:
"L2", mark: 6 } ] },
  { student: "Ali", scores: [ { lab: "L1", mark: 7 }, { lab:
"L2", mark: 8 } ] },
  { student: "Mona", scores: [ { lab: "L1", mark: 10 }, { lab:
"L2", mark: 9 } ] }
])

db.labs.find(
  { scores: { $elemMatch: { lab: "L2", mark: { $gte: 8 } } } },
  { student: 1, _id: 0 }
)
```

mongosh

```
MET> db.labs.find({ scores: { $elemMatch: { lab: "L2", mark: { $gte:
8 } } } }, { student: 1, _id: 0 })
[ { student: 'Ali' }, { student: 'Mona' } ]

MET> db.labs.find({ "scores.lab": "L2", "scores.mark": { $gte: 8 }
}, { student: 1, _id: 0 })
[ { student: 'Sara' }, { student: 'Ali' }, { student: 'Mona' } ]
```

Example output

Without `$elemMatch` the two conditions can match different items, so Sara (L1 = 9, L2 = 6) appears by mistake. `$elemMatch` forces both on the same item.

SECTION 4

Delete documents

PRACTICE ON A TEMP COLLECTION

```
db.temp.insertMany([
  { n: 1, done: true },
  { n: 2, done: true },
  { n: 3, done: false }
])

db.temp.deleteOne({ n: 3 })

db.temp.deleteMany({ done: true })
```

mongosh

```
MET> db.temp.deleteOne({ n: 3 })
{ acknowledged: true, deletedCount: 1 }
MET> db.temp.deleteMany({ done: true })
{ acknowledged: true, deletedCount: 2 }
```

Example output

`deleteOne` removes the first match. `deleteMany` removes every match. There is no undo, and `deleteMany({})` removes everything.

SECTION 4

Update documents: $set, $inc, $unset

COMMANDS

```
db.students.updateOne(
  { name: "Omar Said" },
  { $set: { gpa: 3.0, graduated: false } }
)

db.students.updateOne(
  { name: "Hana Mostafa" }, { $inc: { age: 1 } }
)

db.students.updateOne(
  { name: "Omar Said" }, { $unset: { graduated: "" } }
)
```

mongosh

```
MET> db.students.updateOne({ name: "Omar Said" }, { $set: { gpa:
3.0, graduated: false } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Example output

Operator	Does
`$set`	change a field or add it
`$inc`	add a number to a field
`$unset`	remove a field

SECTION 4

Update many and update arrays

COMMANDS

```
db.students.updateMany(
  { major: "IT" },
  { $set: { faculty: "Computing" } }
)

db.students.updateOne({ name: "Omar Said" },
  { $push: { courses: "Cloud" } })

db.students.updateOne({ name: "Omar Said" },
  { $push: { courses: { $each: ["Linux", "Docker"] } } })

db.students.updateOne({ name: "Omar Said" },
  { $pull: { courses: "Linux" } })
```

mongosh

```
MET> db.students.updateMany({ major: "IT" }, { $set: { faculty:
"Computing" } })
{ acknowledged: true, insertedId: null,
  matchedCount: 3, modifiedCount: 3, upsertedCount: 0 }

MET> db.students.findOne({ name: "Omar Said" }, { courses: 1, _id: 0
})
{ courses: [ 'Networks', 'Cloud', 'Docker' ] }
```

Example output

Operator	Does
`$push`	add an item to an array
`$each`	add several items at once
`$pull`	remove an item from an array
`$addToSet`	add only if it is not there yet

SECTION 4

Why we need a driver



Your app
Node.js code

→

Driver
npm package: mongodb

→

mongod
port 27017

→

Data
on disk

Driver

A library that lets your language talk to MongoDB

Official drivers

Node.js, Python, Java, C#, Go and more

Our stack

Node.js + Express + mongodb driver

SECTION 4

How the pieces fit together

Postman
sends a request

→

Express app
routes the request

→

mongodb driver
talks to the database

→

MongoDB
stores the data

HTTP method	Route	MongoDB operation
GET	/students	`find()`
GET	/students/:id	`findOne()`
POST	/students	`insertOne()`
PATCH	/students/:id	`updateOne()`
DELETE	/students/:id	`deleteOne()`

SECTION 4

Set up the project

1. Install Node.js and check `node --version`
2. Create a folder `students-api` and open a terminal in it
3. `npm init -y` creates package.json
4. `npm install express mongodb` installs the web framework and the driver

TERMINAL

```
node --version
npm init -y
npm install express mongodb
```

terminal

```
$ npm install express mongodb
added 70 packages, and audited 71 packages in 4s
found 0 vulnerabilities
```

Example output

SECTION 4

Project structure in the editor



1. Open the folder `students-api` in your code editor
2. Create `db.js` for the connection and `index.js` for the routes
3. `package.json` and `node_modules` are created by npm
4. Run the app from the terminal: `node index.js`

SECTION 4

Connect to MongoDB: db.js

db.js

```
const { MongoClient } = require("mongodb");

let dbConnection;

module.exports = {
  connectToDb: (cb) => {
    MongoClient.connect("mongodb://localhost:27017/MET")
      .then((client) => {
        dbConnection = client.db();
        return cb();
      })
      .catch((err) => {
        console.log(err);
        return cb(err);
      });
  },
  getDb: () => dbConnection,
};
```

1. `connectToDb` opens the connection and keeps it in `dbConnection`
2. `getDb` gives the rest of the app access to the database
3. The database name `MET` is part of the connection string

> Create the file next to package.json. The server must be running.

SECTION 4

Start the app: index.js

index.js

```
const express = require("express");
const { connectToDb, getDb } = require("./db");

const app = express();
app.use(express.json());

let db;
connectToDb((err) => {
  if (!err) {
    app.listen(3000, () => {
      console.log("app listening on port 3000");
    });
    db = getDb();
  }
});
```

terminal

```
$ node index.js
app listening on port 3000
```

Example output

The app starts listening only after the database connection succeeds. `express.json()` lets Express read JSON sent in a request body.

SECTION 4

Cursors and fetching data: GET /students

index.js · add before app.listen

```
app.get("/students", (req, res) => {
  let students = [];
  db.collection("students")
    .find()
    .sort({ name: 1 })
    .forEach((student) => students.push(student))
    .then(() => res.status(200).json(students))
    .catch(() => res.status(500).json({ error: "Could not fetch
the documents" }));
});
```

Postman

GET `http://localhost:3000/students` Send

Response

200 OK

```
[
  { "_id": "65f1...c002", "name": "Ali Hassan", "gpa": 3.2, ... },
  { "_id": "65f1...c007", "name": "Hana Mostafa", "gpa": 3.1, ... },
  ...
]
```

find() → batch 1 batch 2 batch 3 → forEach / toArray

A cursor hands over documents in batches, so a huge collection never loads all at once.

SECTION 4

Find one document: GET /students/:id

index.js

```
const { ObjectId } = require("mongodb");

app.get("/students/:id", (req, res) => {
  if (ObjectId.isValid(req.params.id)) {
    db.collection("students")
      .findOne({ _id: new ObjectId(req.params.id) })
      .then((doc) => res.status(200).json(doc))
      .catch(() => res.status(500).json({ error: "Could not
fetch the document" }));
  } else {
    res.status(500).json({ error: "Not a valid doc id" });
  }
});
```

Postman

GET `http://localhost:3000/students/65f1a2b3c4d5e6f7a8b9c001` Send

Response

200 OK

```
{
  "_id": "65f1a2b3c4d5e6f7a8b9c001",
  "name": "Sara Ahmed",
  "gpa": 3.6
}
```

The id in the URL is text. `new ObjectId(...)` turns it into the real `_id` type.

SECTION 4

Using Postman

1. Install Postman and open a new request
2. Choose the method (GET, POST, PATCH, DELETE)
3. Type the URL: `http://localhost:3000/students`
4. Click **Send** and read the status and the response

> A browser can only send GET easily. Postman lets us test every method before we build a front end.



SECTION 4

POST: add a document



index.js

```
app.post("/students", (req, res) => {
  const student = req.body;

  db.collection("students")
    .insertOne(student)
    .then((result) => res.status(201).json(result))
    .catch(() => res.status(500).json({ error: "Could not create
a new document" }));
});
```

Postman

POST `http://localhost:3000/students` Send

Body · raw · JSON

```
{
  "name": "Laila Samir", "age": 20, "major": "IT", "gpa": 3.3
}
```

Response

201 Created

```
{
  "acknowledged": true,
  "insertedId": "65f1a2b3c4d5e6f7a8b9c0aa"
}
```

SECTION 4

DELETE: remove a document



index.js

```
app.delete("/students/:id", (req, res) => {
  if (ObjectId.isValid(req.params.id)) {
    db.collection("students")
      .deleteOne({ _id: new ObjectId(req.params.id) })
      .then((result) => res.status(200).json(result))
      .catch(() => res.status(500).json({ error: "Could not
delete the document" }));
  } else {
    res.status(500).json({ error: "Not a valid doc id" });
  }
});
```

Postman

DELETE | http://localhost:3000/students/65f1a2b3c4d5e6f7a8b9c0aa | Send

Response

200 OK

```
{ "acknowledged": true, "deletedCount": 1 }
```

SECTION 4

PATCH: update a document



index.js

```
app.patch("/students/:id", (req, res) => {
  const updates = req.body;

  if (ObjectId.isValid(req.params.id)) {
    db.collection("students")
      .updateOne({ _id: new ObjectId(req.params.id) }, { $set:
updates })
      .then((result) => res.status(200).json(result))
      .catch(() => res.status(500).json({ error: "Could not
update the document" }));
  } else {
    res.status(500).json({ error: "Not a valid doc id" });
  }
});
```

Postman

PATCH | http://localhost:3000/students/65f1a2b3c4d5e6f7a8b9c001 | Send

Body · raw · JSON

```
{ "gpa": 3.8 }
```

Response

200 OK

```
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1
}
```

PATCH changes only the fields you send, using `$set`.

SECTION 4

Pagination

index.js · replace the GET /students route

```
app.get("/students", (req, res) => {
  const page = req.query.p || 0;
  const perPage = 3;
  let students = [];

  db.collection("students")
    .find()
    .sort({ name: 1 })
    .skip(page * perPage)
    .limit(perPage)
    .forEach((s) => students.push(s))
    .then(() => res.status(200).json(students));
});
```

Postman

GET `http://localhost:3000/students?p=1` Send

Response

200 OK

```
[
  { "name": "Mona Khaled", ... },
  { "name": "Nour Adel", ... },
  { "name": "Omar Said", ... }
]
```

?p=0	?p=1	?p=2
Ali · Hana · Karim	Mona · Nour · Omar	Sara · Youssef

skip = page x 3, limit = 3

SECTION 4

Indexes: why they matter

Without an index

MongoDB reads every document to find a match (**COLLSCAN**)

With an index

MongoDB jumps straight to the matching entries (**IXSCAN**)

Like a book

The index at the back tells you the page, so you skip reading the whole book

COLLSCAN

check document 1, 2, 3 ... 1,000,000

IXSCAN

sorted index: jump to gpa 3.9

Trade-off: indexes make reads faster but take space and make writes a little slower. Index the fields you search or sort by often.

SECTION 4

Indexes in the shell

COMMANDS

```
db.students.createIndex({ gpa: 1 })

db.students.getIndexes()

db.students.find({ gpa: 3.9 }).explain("executionStats")

db.students.dropIndex("gpa_1")
```

mongosh

```
MET> db.students.createIndex({ gpa: 1 })
gpa_1
MET> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { gpa: 1 }, name: 'gpa_1' }
]
MET> db.students.find({ gpa: 3.9 }).explain("executionStats")
{ ... winningPlan: { stage: 'FETCH', inputStage: { stage: 'IXSCAN' }
} ... }
```

Example output

Documents | Aggregations | Schema | **Indexes 2** | Validation

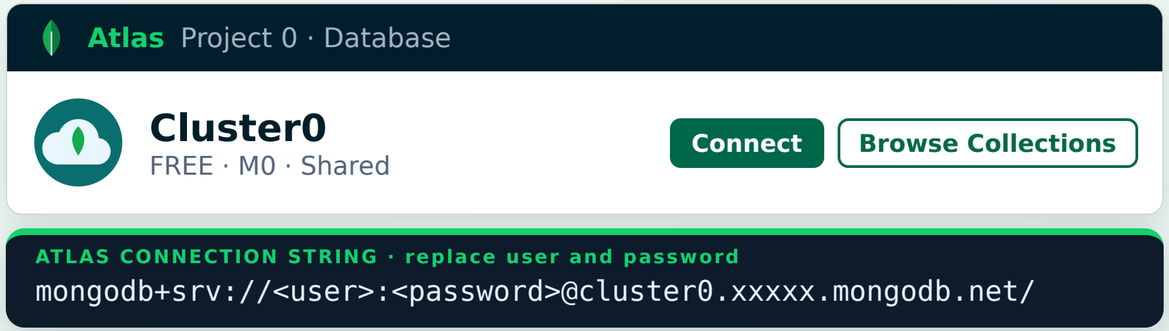
Name and definition	Type	Size	Usage	Properties
_id_ { _id: 1 }	REGULAR	36 KB	0	UNIQUE
gpa_1 { gpa: 1 }	REGULAR	4 KB	0	-

Compass: the Indexes tab of the students collection

SECTION 4

MongoDB Atlas: MongoDB in the cloud

1. Create a free account at mongodb.com/atlas
2. Create a free cluster (M0)
3. Database Access: add a database user with a password
4. Network Access: add your IP address
5. Click **Connect**, choose Drivers or Compass, copy the connection string



SECTION 4

Use Atlas in Compass and in the app



.env file

```
MONGO_URI=mongodb+srv://user:password@cluster0.xxxxx.mongodb.net/MET
```

db.js · only this line changes

```
// db.js
MongoClient.connect(process.env.MONGO_URI)
```

> Never put the real password in code you share. Keep it in a .env file and out of Git.

1. Compass: **Add new connection**, paste the Atlas string, Save & Connect
2. App: replace the local string, run `node index.js`
3. Same code, same queries: only the address changed

SECTION 4

Cheat sheet: Compass, shell and Node

Task	Compass	mongosh	Node driver
Create	ADD DATA	`insertOne` · `insertMany`	`insertOne(doc)`
Read	filter bar + Find	`find` · `findOne`	`find().toArray()`
Update	pencil + UPDATE	`updateOne` · `updateMany`	`updateOne(f, { $set })`
Delete	trash + DELETE	`deleteOne` · `deleteMany`	`deleteOne(filter)`
Sort / limit	Options	`.sort()` · `.limit()` · `.skip()`	same chain
Index	Indexes tab	`createIndex`	`createIndex`

SECTION 4

Key takeaways



mongosh

Same operations as Compass, typed as commands

Operators

`$gt $or $in $all $elemMatch $set $push`

Node API

Driver + Express: GET, POST, PATCH, DELETE

Scale

Pagination with skip and limit, indexes for speed, Atlas for the cloud

You can now store, query, update and serve data from MongoDB.

BIG DATA COURSE

SECTION 4

Thank you

Prepared by: Amira Mahmoud

